

L8 Colab 数据分析综合练习题集

这两道题继续在 **Google Colab** 上完成。操作步骤：

1. 打开 [Google Colab](#)
2. 新建一个 Notebook
3. 把数据文件上传到 Colab（点击左侧文件夹图标，然后上传）
4. 在代码格里写代码，运行看结果

完成后把你的 **Colab 链接** 放到 `submission` 文件夹下。

这次我们不再只做天气分析，而是换成一个更接近 UNSW 数据分析课程语境的数据：**学生课程成绩数据**。

UNSW 公开课程页面里可以看到一些和数据分析相关的课程方向，例如：

- `DATA1001`：Introduction to Data Science and Decisions
- `COMM1190`：Data, Insights and Decisions
- `COMM2501`：Data Visualisation and Communication

公开页面没有给出固定 CSV 数据集，所以这里用一个模拟的学生成绩表 `student_course_results.csv`。这类数据在数据分析入门里很常见：一行代表一名学生在一门课的一次成绩记录。

数据文件有 7 列：

列名	含义
<code>student_id</code>	学生编号
<code>faculty</code>	学院，例如 Science、Business、Engineering
<code>program</code>	专业
<code>course</code>	课程编号，例如 DATA1001、COMM1190
<code>term</code>	学期，例如 T1、T2、T3
<code>mark</code>	成绩
<code>attendance</code>	出勤率

Exercise 1: 各课程平均成绩 ★★

计算每门课程的平均成绩，按课程编号字母顺序输出。

这道题要求用两种方法完成：

1. **Method 1:** 使用 Python 原生方法 `with open` 读取
2. **Method 2:** Use package，也就是用 pandas 的简洁语法实现

两种方法的输出必须完全一样。做两遍的目的不是重复劳动，而是先把基础的文件读取、标题行处理、逐行拆字段、字典分组这些逻辑理顺，再理解 pandas 的 `groupby().mean()` 到底帮你省掉了哪些步骤。

Method 1: 使用 Python 原生方法 `with open` 读取

手动处理每一行和标题行：跳过标题行，用 `split(',')` 拆字段，用字典收集每门课的所有成绩，最后手动算平均值。

要求：

1. 用 `with open("student_course_results.csv") as f` 打开文件
2. 用 `next(f)` 跳过第一行标题
3. 每一行用 `strip()` 去掉末尾换行，再用 `split(",")` 拆字段
4. 取出 `course`（第 4 列，下标 3）和 `mark`（第 6 列，下标 5）
5. 把 `mark` 转成 `float`
6. 用一个字典，key 是课程编号，value 是这门课所有成绩的列表
7. 如果这门课第一次出现，先创建空列表；否则往列表里 `append`
8. 遍历字典，对每门课计算平均成绩：`sum(列表) / len(列表)`
9. 按课程编号字母顺序输出：`课程编号 : 平均成绩`
10. 平均成绩保留 1 位小数

测试代码 + 预期输出

```
# COMM1190 : 72.6
# COMM2501 : 74.8
# COMP1010 : 65.3
# DATA1001 : 70.8
```

代码框架

```
marks_by_course = {}

with open("student_course_results.csv") as f:
    next(f) # 跳过标题行
    for row in f:
        values = row.strip().split(",")
```

```
# 取出 course 和 mark
course = None
mark = None

# 如果这门课第一次出现，先创建空列表
# 然后把 mark 追加进去
pass

# 按课程编号字母顺序输出
for course in sorted(marks_by_course):
    mark_list = marks_by_course[course]
    avg = sum(mark_list) / len(mark_list)
    print(course, ":", round(avg, 1))
```

Method 2: Use package, 用 pandas 简洁语法实现

同样的结果，用 pandas 可以更短：`read_csv` → `groupby` → `.mean()`。

要求：

1. 导入 `pandas as pd`
2. 用 `pd.read_csv()` 读取 `student_course_results.csv`
3. 用 `course` 分组
4. 对 `mark` 求 `.mean()`
5. 按课程编号字母顺序输出：`课程编号 : 平均成绩`
6. 平均成绩保留 1 位小数

测试代码 + 预期输出

```
# COMM1190 : 72.6
# COMM2501 : 74.8
# COMP1010 : 65.3
# DATA1001 : 70.8
```

代码框架

```
import pandas as pd

df = pd.read_csv("student_course_results.csv")

# 按 course 分组, 对 mark 求平均值
avg_mark = None

# 按课程编号字母顺序输出
for course in sorted(avg_mark.index):
    print(course, ":", round(float(avg_mark[course]), 1))
```

对比两种方法

	Method 1 <code>with open</code>	Method 2 <code>pandas</code>
读文件	手动 <code>open</code> 、 <code>next(f)</code> 跳标题、 <code>split</code> 拆字段	<code>pd.read_csv()</code> 一行搞定
分组	自己用字典, <code>if course not in dict</code> 判断	<code>groupby('course')</code>
求平均	<code>sum(list) / len(list)</code>	<code>.mean()</code>
排序	<code>sorted(dict)</code>	<code>sorted(series.index)</code>
优点	看得清每一步	简洁、适合真实数据分析

Method 1 是为了理解数据分析的基础流程；Method 2 是为了把同样的逻辑写得更简洁。

Exercise 2: 各学院高表现学生统计 ★★★

定义：如果一条记录同时满足下面两个条件，就算一条“高表现记录”：

- `mark >= 85`
- `attendance >= 90`

你需要按学院统计：

- 每个学院有多少条高表现记录
- 这些高表现记录的平均成绩是多少

这题综合了前面学过的 `pandas` 技巧：布尔筛选、多个条件、`groupby`、`.size()`、`.mean()`、字典 `.get()`。

要求：

- 导入 `pandas as pd`

2. 读取 `student_course_results.csv`
3. 获取数据中所有学院名，并按字母顺序排序
4. 用两个条件筛选出高表现记录： `mark >= 85` 并且 `attendance >= 90`
5. 对高表现记录按 `faculty` 分组，用 `.size()` 统计每个学院的记录数
6. 对高表现记录按 `faculty` 分组，用 `.mean()` 算每个学院高表现记录的平均成绩
7. 把两个结果都转成字典
8. 遍历所有学院名，输出：学院：X students, avg mark Y
9. 如果某个学院没有高表现记录，输出 0 students, avg mark N/A
10. 平均成绩保留 1 位小数

测试代码 + 预期输出

```
# Business : 4 students, avg mark 90.2
# Engineering : 1 students, avg mark 94.0
# Science : 2 students, avg mark 89.0
```

代码框架

```
import pandas as pd

df = pd.read_csv("student_course_results.csv")

# 获取所有学院名，排好序
all_faculties = sorted(df["faculty"].unique())

# 布尔筛选：成绩 >= 85 且 出勤率 >= 90
high_records = None

# 对高表现记录按 faculty 分组，统计条数
count_dict = None

# 对高表现记录按 faculty 分组，算平均成绩
avg_mark_dict = None

# 遍历所有学院名，输出
for faculty in all_faculties:
    count = count_dict.get(faculty, 0)
    if count > 0:
        avg_mark = round(avg_mark_dict[faculty], 1)
        print(f"{faculty} : {count} students, avg mark {avg_mark}")
    else:
        print(f"{faculty} : 0 students, avg mark N/A")
```

这题在练什么

1. 多个 pandas 条件要分别加括号：(条件1) & (条件2)
2. 布尔筛选先缩小范围，再做分组统计
3. 同一个筛选结果可以做不同聚合：.size() 数数量，.mean() 算平均
4. 真实数据分析里经常要先定义“好学生”“高风险学生”“高价值客户”等规则，再按组统计

提交说明

完成后，在 submission 文件夹下新建一个文件（比如 你的名字_L9.txt），把你的 Colab Notebook 链接粘贴进去。

Exercise 3 : LeetCode 练习：链表遍历

完成 双指针技巧秒杀链表 的阅读

<https://labuladong.online/zh/algo/essential-technique/linked-list-skills-summary/>

在 LeetCode 上完成以下 3 个练习，并截图提交：

88. 合并两个有序数组

21. 合并两个有序链表

141. 环形链表